University of Applied Sciences and Arts Northwestern Switzerland
School of Business

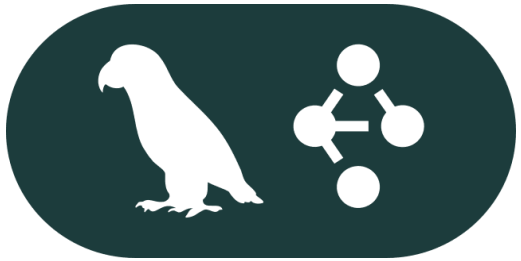
member of
swissuniversities

AACSB
ACCREDITED

Generative KI und Agentensysteme

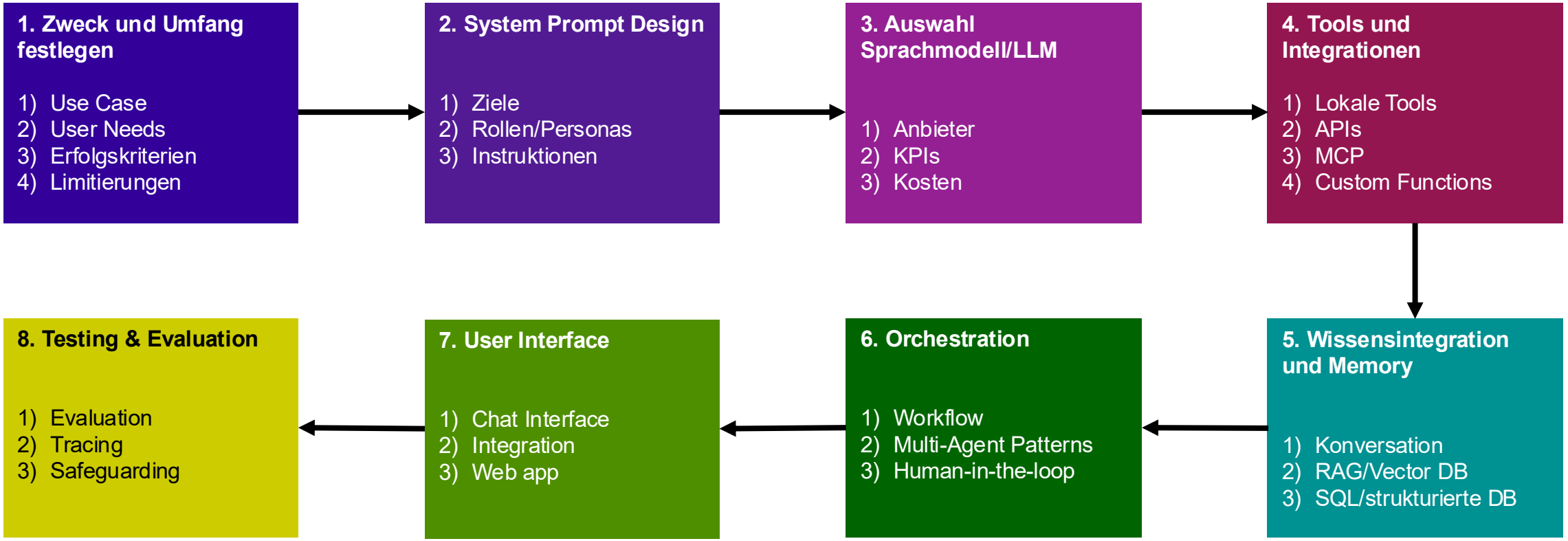
SW4

Agentensystem: Bausteine, Vorgehen, Modellauswahl



Sandro Schwander

Ziel und Inhalte: die Bausteine eines Agentensystems



Grundsätze Projekt und Unterrichtsblock 2

- Der genaue Projektbeschrieb ist auf Moodle.
  - Einen eigenen LLM-Agenten entwickeln. Dies wird euch während der nächsten 6 Wochen schrittweise erklärt und gezeigt anhand von **LangGraph**. Wichtig: Wir unterrichten einen „pro-code“ Approach, d.h. no- bzw. low-code Tools wie n8n, voiceflow und andere sollen in diesem Modul nicht genutzt werden.
  - Alternative 1: Anpassung eines LLMs mittels Post-Training, die Entwicklung oder Evaluation von Embedding- und Retrieval-Verfahren oder eine systematische Untersuchung unterschiedlicher LLMs. Dies muss mit den Dozierenden abgestimmt sein.
  - Alternative 2: Apertus und Hack Apertus. Einsatz von Apertus als LLM innerhalb eines Agenten oder einer LLM-Anwendung;
    - Vergleich von Apertus mit anderen offenen oder kommerziellen LLMs;
    - Anpassung von Apertus mittels Supervised Fine-Tuning oder anderer Post-Training-Verfahren;
    - Untersuchung von Retrieval-, Embedding- oder RAG-Ansätzen im Zusammenspiel mit Apertus;
    - Evaluation von Qualität, Robustheit, Bias, Guardrails, Tool-Use oder anderen relevanten Eigenschaften.

Projekt

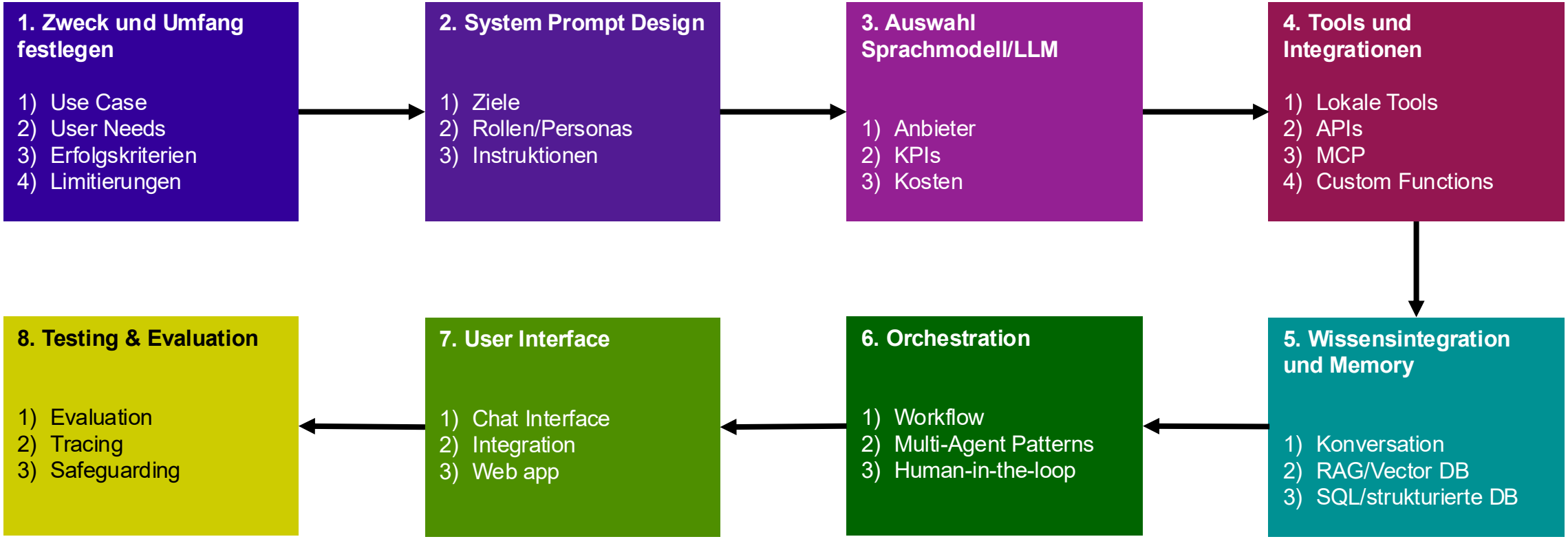
- Ziel: wird überwiegend während der Unterrichtszeit erarbeitet
- Teams aus 3 Studierenden, max. 4
- KW48 (23.11.26, 13.00 Uhr) findet eine Abschlusspräsentation statt (10 Min. inkl. Verständnisfragen)

Ablauf pro Woche

- Theorie-Block (ca. 2 Lektionen)
  - Gruppen erarbeiten direkt Inhalte dazwischen.
- Demo/Beispiel anhand von LangGraph (ca. 1 Lektion)
- Zeit die Demo selbst anzuwenden/anzupassen, Erfahrungen zu sammeln und am Projekt zu arbeiten. (ca. 1 Lektion)

Die Praxisblöcke mit LangGraph sind möglichst aufeinander aufbauend und folgen den Bausteinen wie oben beschrieben.

Vom Chatbot zum Agenten: Was in welcher Woche dazu kommt



Every Group Project:



Semesterwoche 4

SW4: Übersicht und Inhalte

1. Was ist ein Agent? Was ist der Unterschied zu einem Chatbot oder LLM?
2. Weshalb existieren LLM-Orchestratoren?
3. Use-Case und Nutzerbedürfnisse: Wie komme ich von einer Idee zu einem funktionierenden Agenten?
4. Erfolgskriterien: Woran messen wir Erfolg bei Agentensystemen? Wie kann man Erfolg messbar machen?
5. Sprachmodell: Wie kann man ein Sprachmodell auswählen?

https://docs.langchain.com/oss/python/langgraph/thinking-in-langgraph

Agenten – Definition

Ein **Agent** ist etwas, was mit Hilfe von **Sensoren** seine Umgebung wahrnimmt und mit Hilfe von **Akt(uat)oren** auf diese Umgebung einwirkt.

- Russell & Norvig, AI: A Modern Approach (2003)

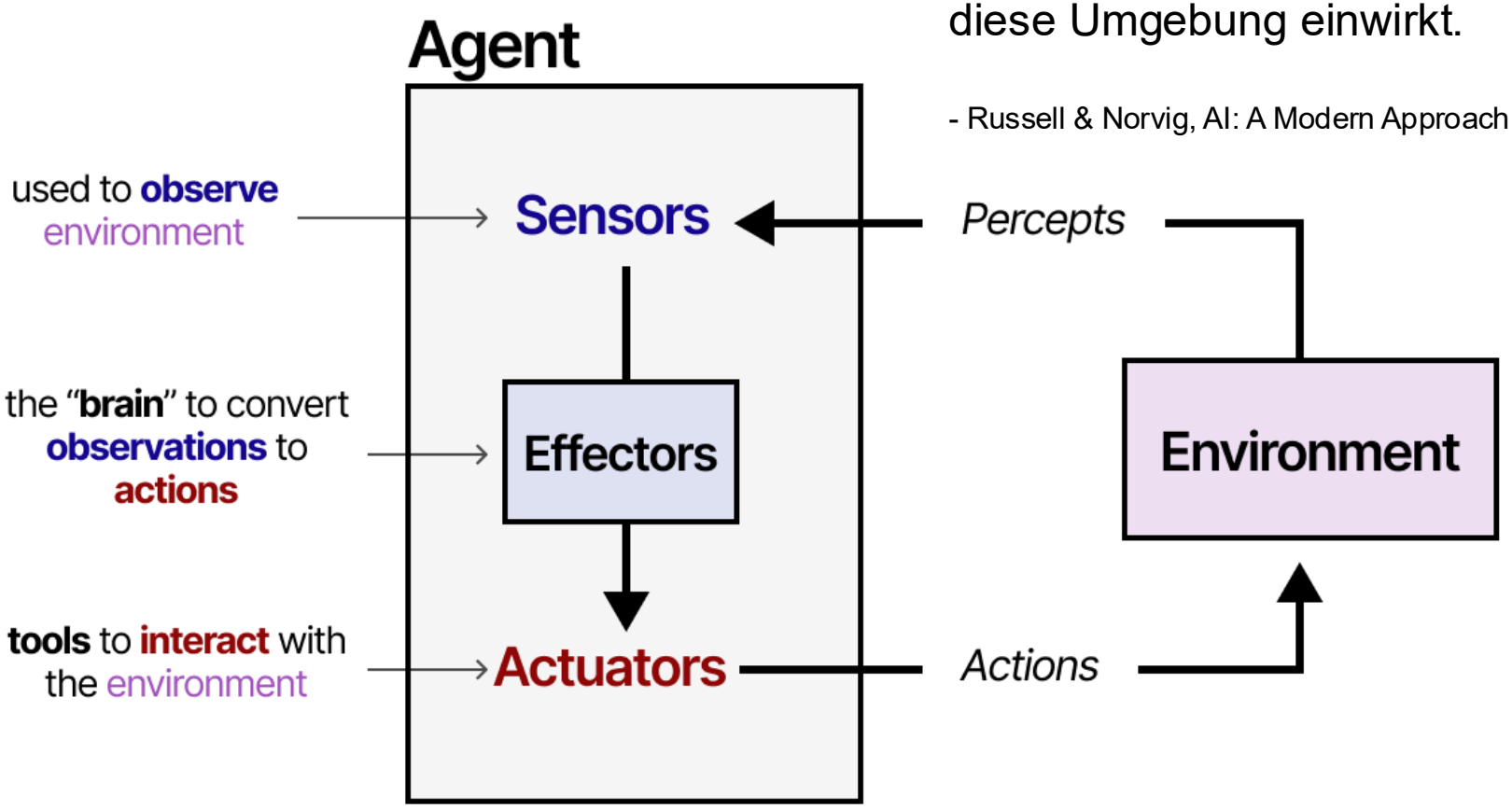
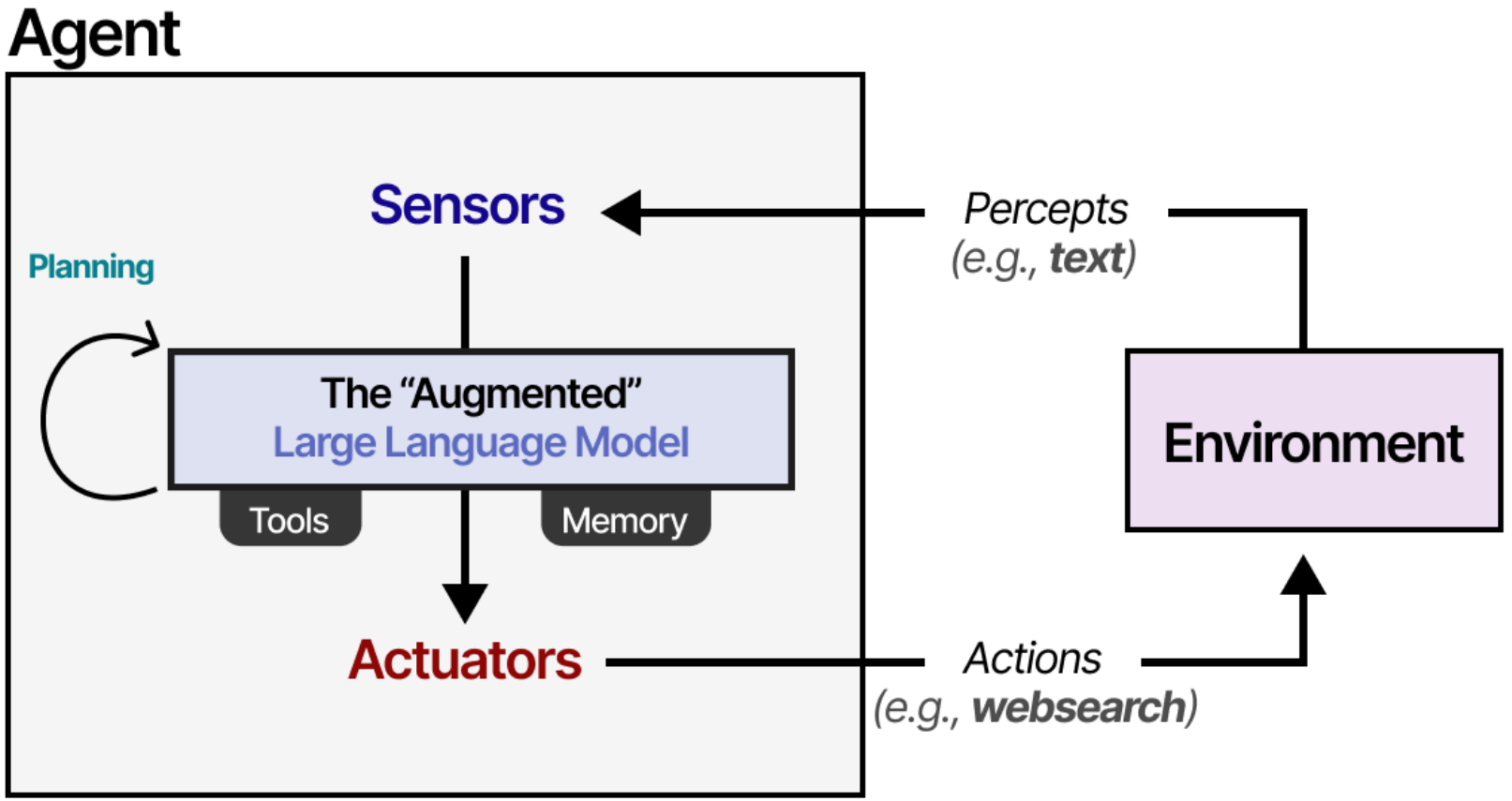


Bild: https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents

The Augmented LLM Agent



- **Wahrnehmung** von z.B. Texteingaben
- Z. B. Ausgabe von **Websuch-Aktionen**
- **Augmented** durch:
  1. *Tools*
  2. *Memory*
- **Autonom** durch:
  3. *Planung*

Reasoning

Bild übernommen von: https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents

Warum?

– Warum existieren LLM-Orchestratoren wie LangChain, Haystack oder LlamaIndex?

1. **Abstraktion und Integration:** Benutzeranfragen in Einzelschritte zerlegen, Prompts strukturieren, mit Gedächtnis arbeiten, Integration in Applikationen, Safeguarding

2. **Retrieval Augmented Generation (RAG):** Verbindung von LLMs mit externem Wissen (Beispielsweise interne Verträge etc.)

3. **Tool-Orchestrierung & Agentenverhalten:** APIs aufrufen, Anfragen weiterleiten, Entscheide innerhalb von Interaktion (Agentenverhalten).

4. **Standardisierung:** Standardisierte Schnittstellen (bspw. für Retreiver, LLMs), Austauschbarkeit und Wiederverwendbarkeit

5. **Probabilistisches und Deterministisches Verhalten steuern:** Definieren, wann ein Sprachmodell zum Einsatz kommt und wann nicht (z.B. feste Antworten, übergabe an Menschen)

Use-Case und Nutzerbedürfnisse

– Beispiel

Der Agent sollte:

- *Eingehende Kunden-E-Mails lesen*
- *Diese nach Dringlichkeit und Thema einordnen*
- *Relevante Unterlagen durchsuchen, um Fragen zu beantworten*
- *Passende Antworten verfassen*
- *Komplexe Probleme an menschliche Mitarbeiter weiterleiten*
- *Bei Bedarf Folgeaktionen planen oder einen Menschen beiziehen*

Der Agent sollte:

- *Eingehende Fragen von Kunden zu Gepäck und Buchung korrekt beantworten*
- *Flugtarife korrekt liefern*
- *Themenfremde Anfragen beispielsweise Nachhaltigkeit des Konzerns etc. erfassen und beantworten können*
- *Bei zu komplexen Fällen an einen Menschen übergeben können*

https://docs.langchain.com/oss/python/langgraph/thinking-in-langgraph

Beispiel: User Stories beschreiben

– Als <User> möchte ich <Aktion/Funktionalität> damit ich <zu erreichendes Ziel>



https://asana.com/de/resources/user-stories

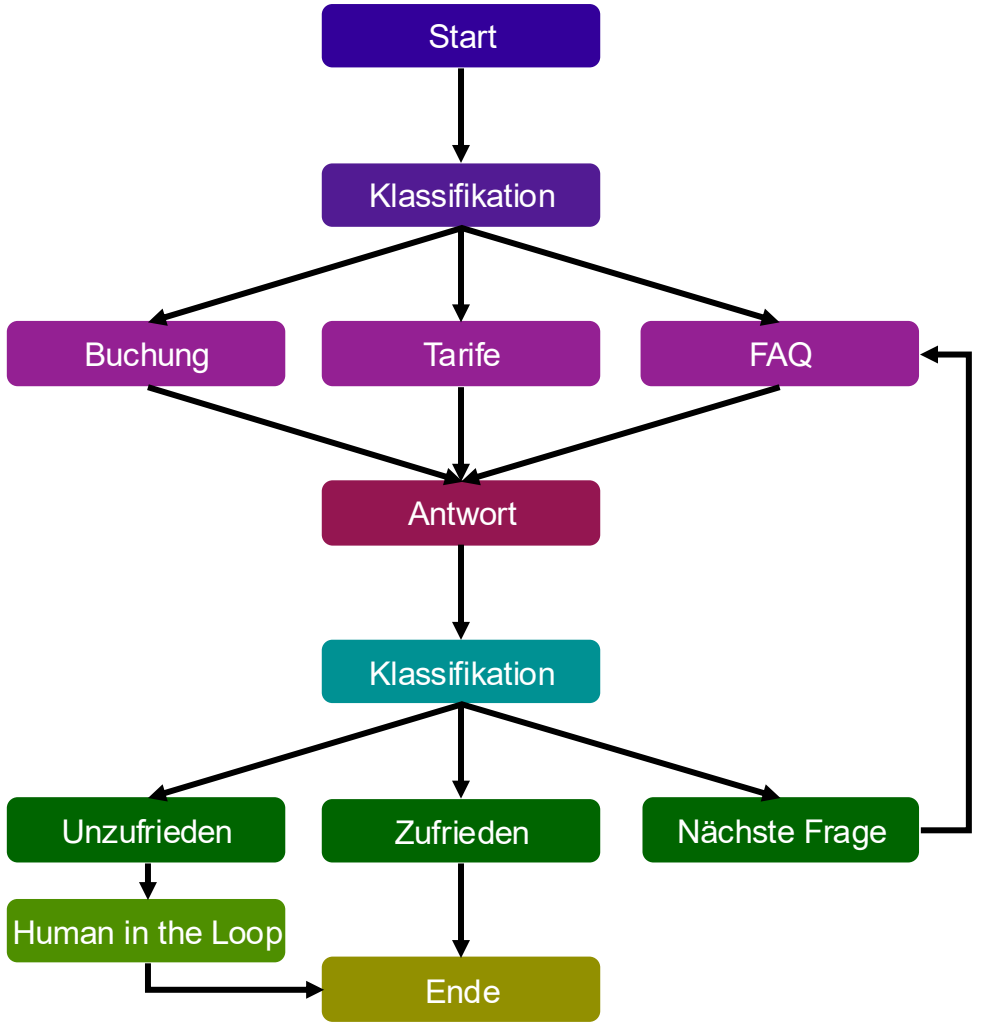
> Gruppenarbeit

1. Wählt ein Thema aus
2. Entscheidet euch für einen Use-Case
3. Schreibt den Use-Case auf, analog zu Slide 14
   - Benutzt User Stories um den Use-Case zu verfeinern
   - Was wollt ihr in den nächsten Wochen umsetzen?

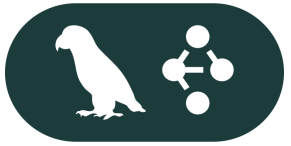
Zeit: 30 Min.

LangGraph Schritt 1: Arbeitsablauf in einzelnen Schritten zerlegen

- Wo wird ein Entscheid gefällt?
- Wann braucht es allenfalls mehrere Iterationen?
- Was ist das Endprodukt?
- Welche separaten Use-Cases gibt es?

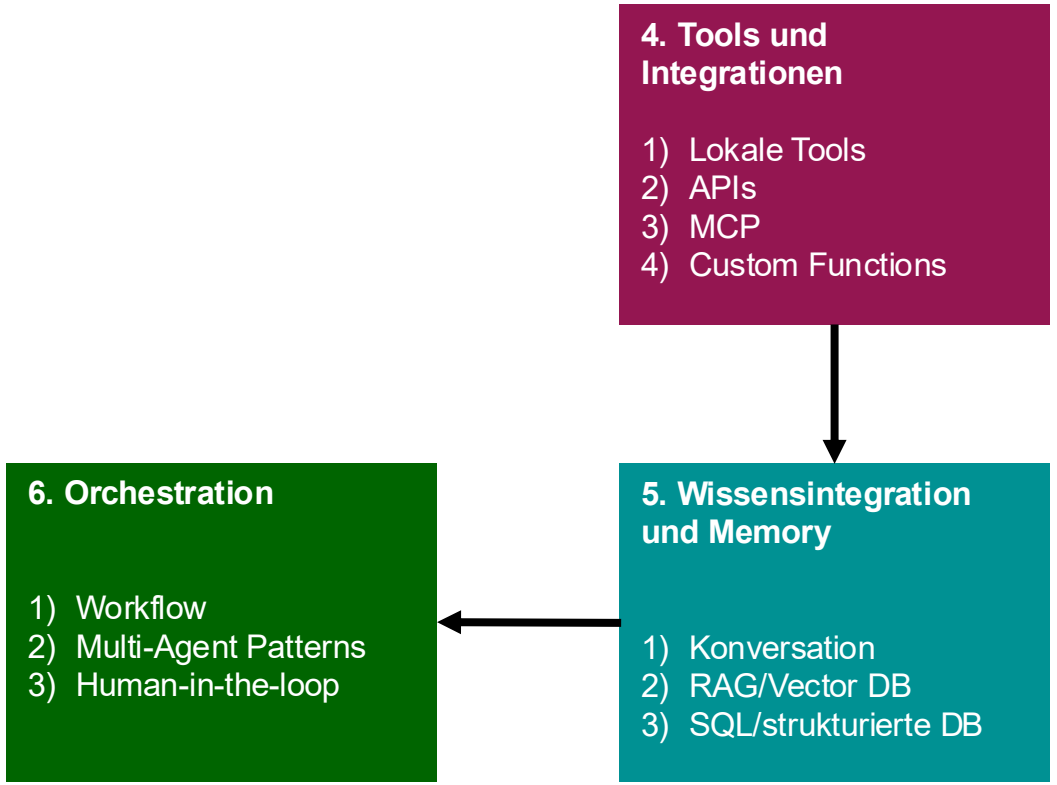


https://docs.langchain.com/oss/python/langgraph/thinking-in-langgraph

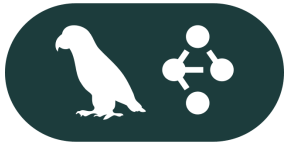


LangGraph Schritt 2: Identifikation von Inhalten und Aufgaben pro Schritt

- Braucht es ein Tool (call)?
- Braucht es eine Wissens-Datenbank?
- Falls ja, strukturiert? Retrieval Augmented Generation (RAG)?
- Welche Orchestration braucht es?
- Braucht es zusätzlichen Input der Nutzer*in?



https://docs.langchain.com/oss/python/langgraph/thinking-in-langgraph



LangGraph Schritt 3: Was braucht es im «Conversation-State»?

- Welche Informationen sollen weitergegeben werden im Prozess?
- Was kann errechnet oder z.B. von Schnittstelle geholt werden?
- Was braucht man nicht?

```python
class EmailAgentState(TypedDict):
    # Raw email data
    email_content: str
    sender_email: str
    email_id: str

    # Classification result
    classification: EmailClassification | None

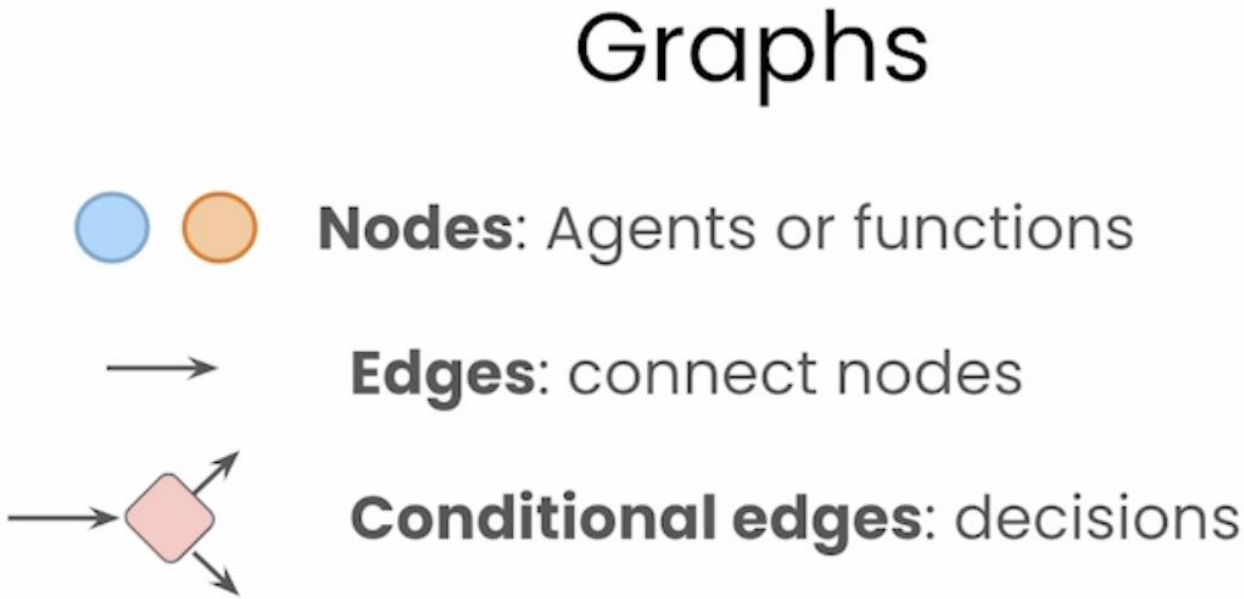
    # Raw search/API results
    search_results: list[str] | None  # List of raw document chunks
    customer_history: dict | None  # Raw customer data from CRM

    # Generated content
    draft_response: str | None
    messages: list[str] | None
```

https://docs.langchain.com/oss/python/langgraph/thinking-in-langgraph

LangGraph Schritt 4+5: Die Nodes erstellen und zusammen verdrahten

- Den Ablauf in Nodes überführen
- Danach mit Edges und Conditional Edges zusammen verbinden



Erfolgskriterien und Limitierungen

- Welches Problem lösen wir?
- Braucht es dafür überhaupt einen Agenten?

- Was ist im „Scope“? Was noch nicht?
- MVP: Produktinkremente

- Was sind unsere Ziele?
- Woran messen wir Erfolg?
- Wie vergleichen wir es mit der heutigen Situation?

- Risiken?
- Einschränkungen (z.B. Datensicherheit, Datenverfügbarkeit, Budget)?



Geronimo Wolf, 2025, https://www.linkedin.com/pulse/verspilling-bij-ict-projecten-binnen-de-overheid-hoe-mvp-denken-wolf-krhye

> Gruppenarbeit

1. Ordnet den Use-Case in einen formalen Prozess
2. Ihr könnt dafür BPMN benutzen oder auch anderweitig den Prozess festhalten
3. Diskutiert, in welchem Prozessschritt welche Anforderungen/Aktionen gebraucht werden → Dies werdet ihr zu einem späteren Zeitpunkt noch verfeinern.
4. Definiert Erfolgskriterien für euer Agentensystem: Wann sprecht ihr von einem Erfolg? -> Diese können auch nur theoretisch sein, falls ihr keinen Use-Case aus der realen Welt benutzt

Zeit: 30 Min.

Modellauswahl

1. Modelleauswahl (1/3): Use-Case bzw. was ist das Problem, das wir lösen möchten?

Kategorie	Unter-Kategorie	Anforderungen
Generating	- Textgenerierung / Chat - Summarization - Paraphrasierung - Codegenerierung-/erklärung	- große Sprachmodelle, gutes Training, verlässlicher Kontextzugriff.
Embedding	- Semantische Suche - RAG - Clustering / Klassifikation	- konsistente Embedding-Modelle, stabile API, große Batch-Verarbeitung.
Multimodalität	Beispielsweise Video+Text, Audio+Text	multimodale Modelle
Multi-Turn/ Reasoning	- Planning - Tool-Use/Function Calling - Orchestrierung - CoT Reasoning	Modelle mit Reasoning und Support für Function Calling

Grundsätzlich: Anwendungsgebiet und Aufgabe

- **General Language Models**: Entwickelt für vielfältige sprachliche Aufgaben wie Textgenerierung, Zusammenfassung und Übersetzung (z. B. GPT-6 Astra, Claude Opus 5).
- **Simulated Reasoning Models**: Optimiert für schrittweises logisches Denken, komplexe Problemlösung und analytische Entscheidungsfindung (z. B. DeepSeek-V4-Pro, Claude Opus 5 mit Extended Thinking).
- **Multimodal Models**: Verarbeiten mehrere Datentypen wie Text, Bilder, Audio und Video, häufig für kreative und unterstützende Aufgaben (z. B. Google Gemini 3).
- **Coding Models**: Spezialisiert auf Softwareentwicklung, fähig zur Codegenerierung, Fehlerbehebung und Erklärung (z. B. Claude Opus 5 / Claude Code, GPT-6 Astra).
- **Domain-Specific Models**: Maßgeschneidert für Bereiche wie Medizin, Recht und Wissenschaft, mit Expertenwissen (z. B. Google MedGemma für medizinische Analysen).

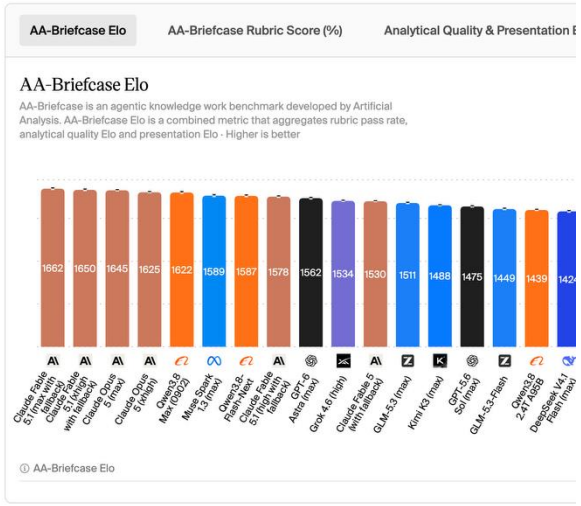
1. **Modellauswahl (2/3):** Sonstige Kriterien

1. **Endnutzer:** Wer sind die Endnutzer, welche Erfahrungen haben sie? Was sind ihre Erwartungen an Genauigkeit, Vertrauenswürdigkeit, Korrektheit?
2. **Throughput & Geschwindigkeit:** Skalierbarkeit, Nutzungsvolumen, Context Window, Latenz (Time-to-First-Token) – Trade-off Intelligenz ↔ Geschwindigkeit
3. **Kosten:** Budget der Applikation, Modellkosten vergleichen (pro Token, Query etc.), OPEX vs. CAPEX, Total Cost of Ownership (Hardware, DevOps)
4. **Technische Anforderungen:** Fine-Tuning-Optionen, Request Limits, Verfügbarkeit/SLA/Redundanz, Response Time
5. **Betriebsmodell & Sonstiges:** API-only vs. lokaler Betrieb, Komplexität der Integration, Dokumentation, Support

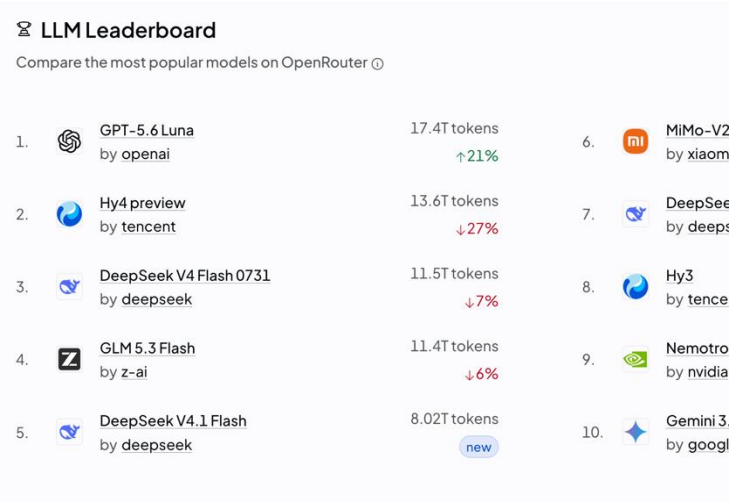
https://coaxsoft.com/blog/llm-api-comparison

Modell Vergleich Beispiel Artificial Analysis und Openrouter

Artificial Analysis:



Openrouter:



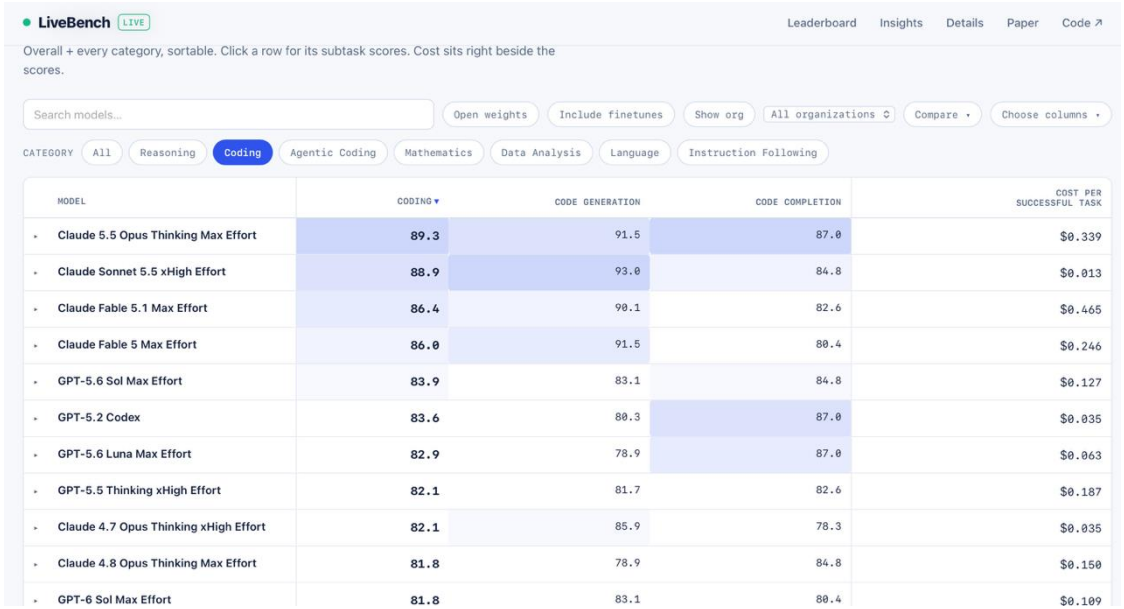
https://artificialanalysis.ai/, https://openrouter.ai/rankings

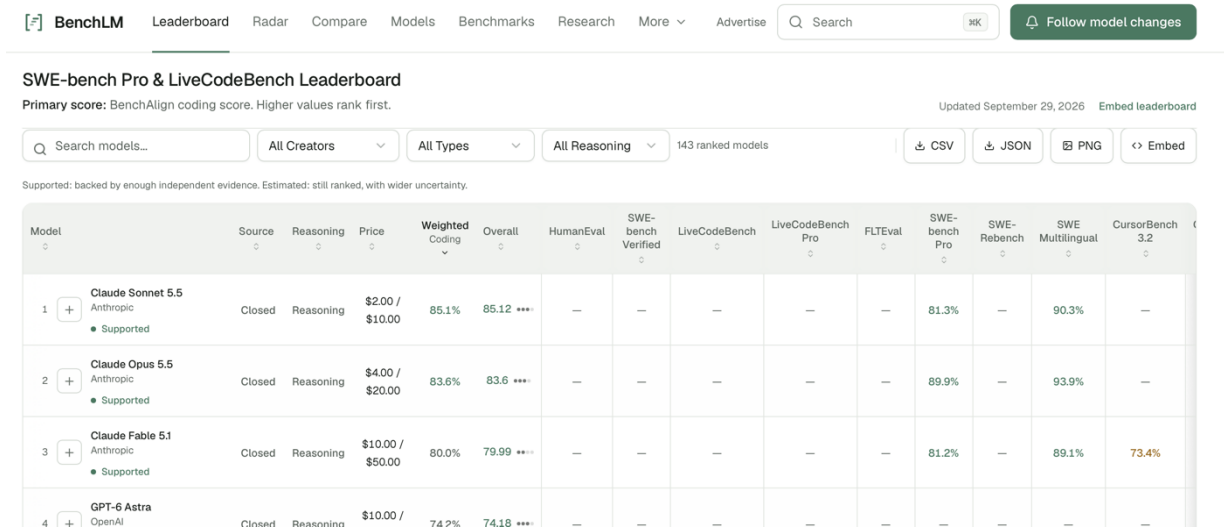
> Aufgabe

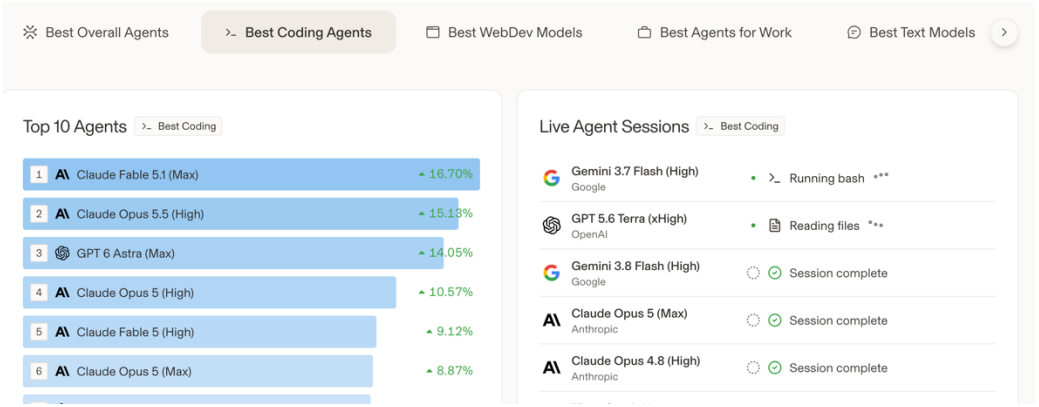
1. Geht auf artificialanalysis.ai
2. Schaut euch die Seite einmal in Ruhe an
3. Sucht nach verschiedenen Kriterien zur Bewertung. Was wird gemessen? Wie wird beispielsweise „Intelligenz“ definiert? Wie werden Bildgenerierungs-Modelle verglichen?
4. Sucht nach Kostenvergleichen. Wie werden Kosten definiert? Was gibt es für unterschiedliche Blickwinkel für die Kostenthematik?

Zeit: 15 Min.

Verschiedene Leaderboards

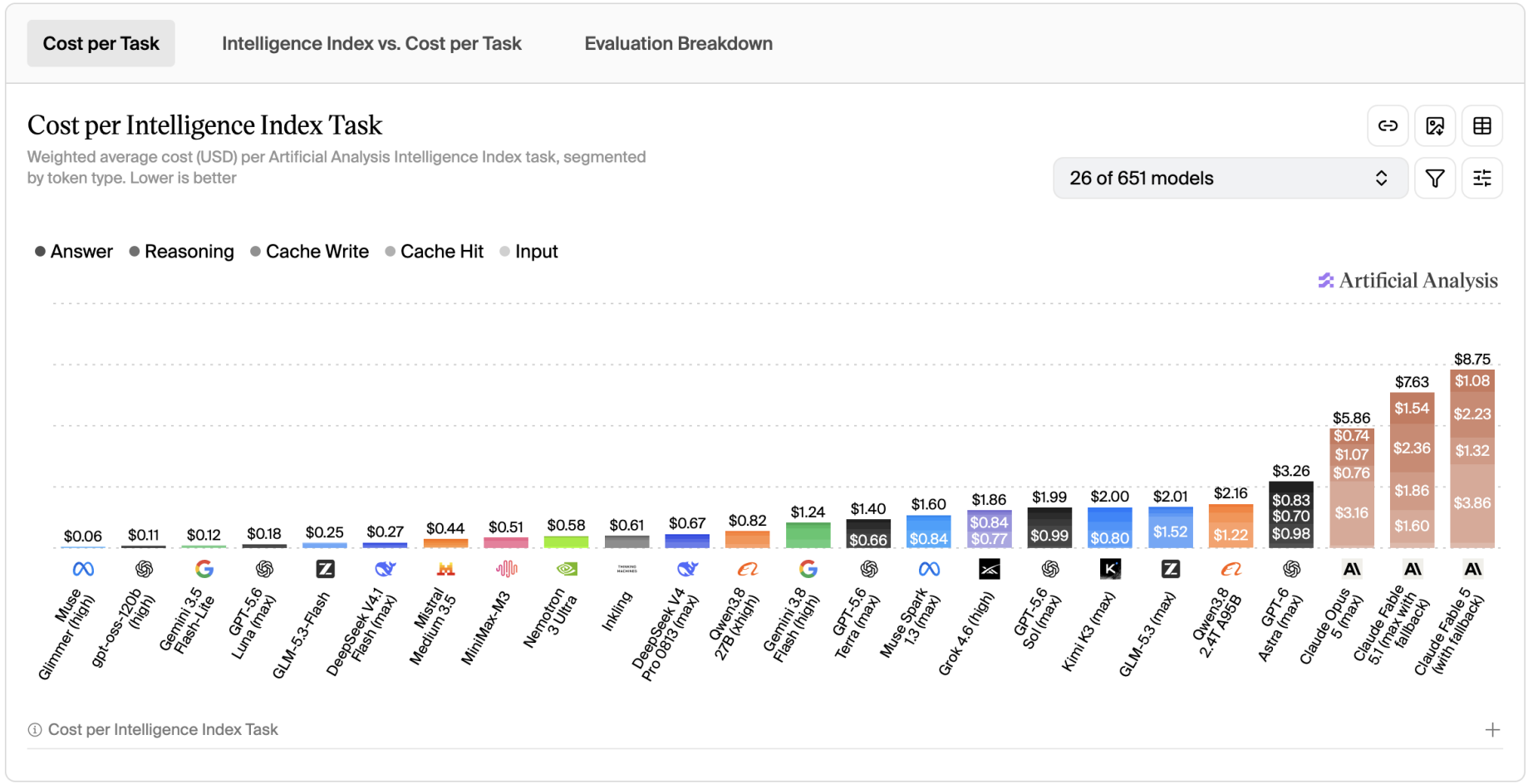






https://livebench.ai/#/?cats=Coding, https://benchlm.ai/coding, https://huggingface.co/spaces/lmarena-ai/arena-leaderboard

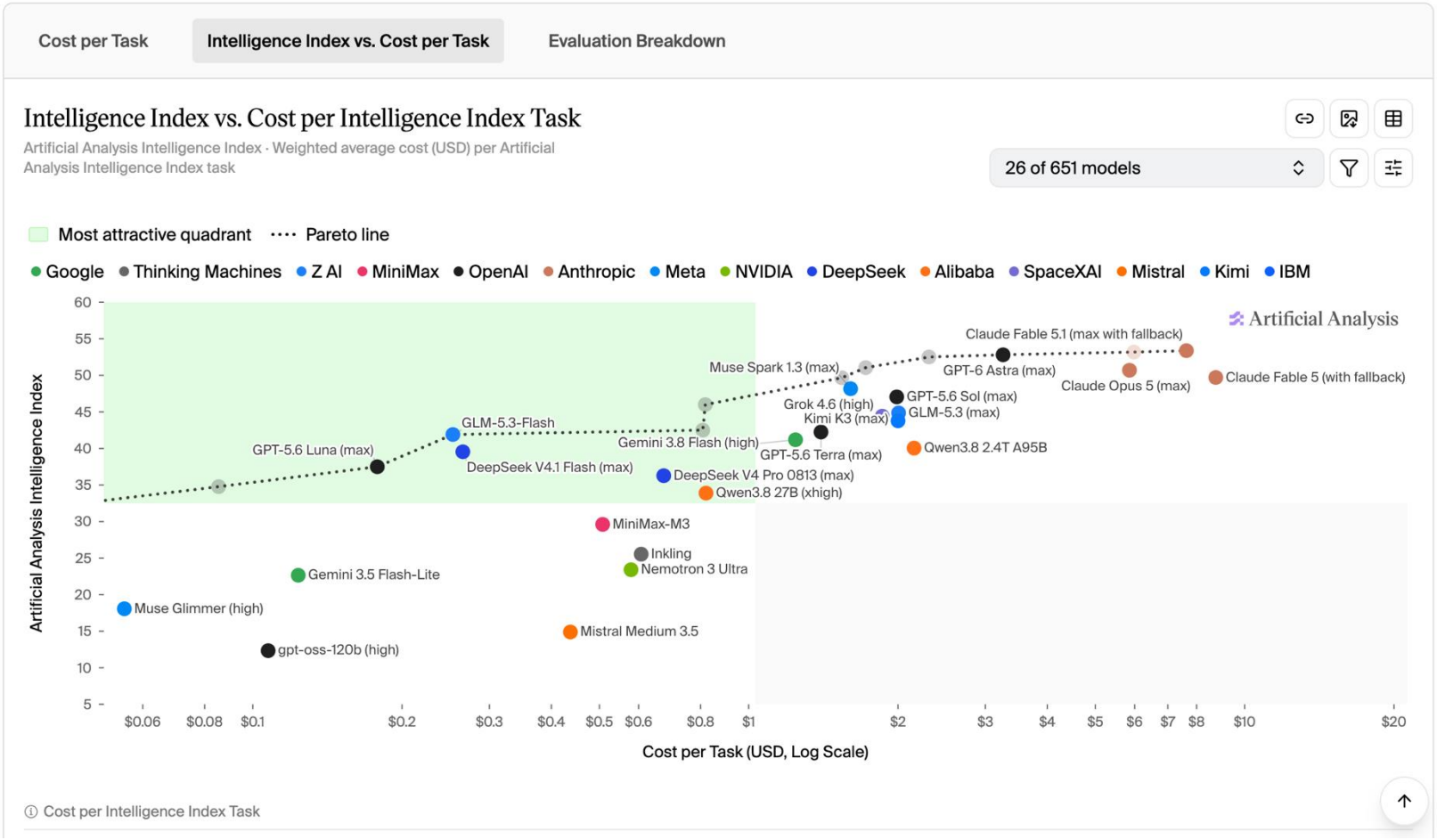
Kosten: Token Sicht



https://artificialanalysis.ai/models#price-cost

Kosten: Preis vs. Intelligenz

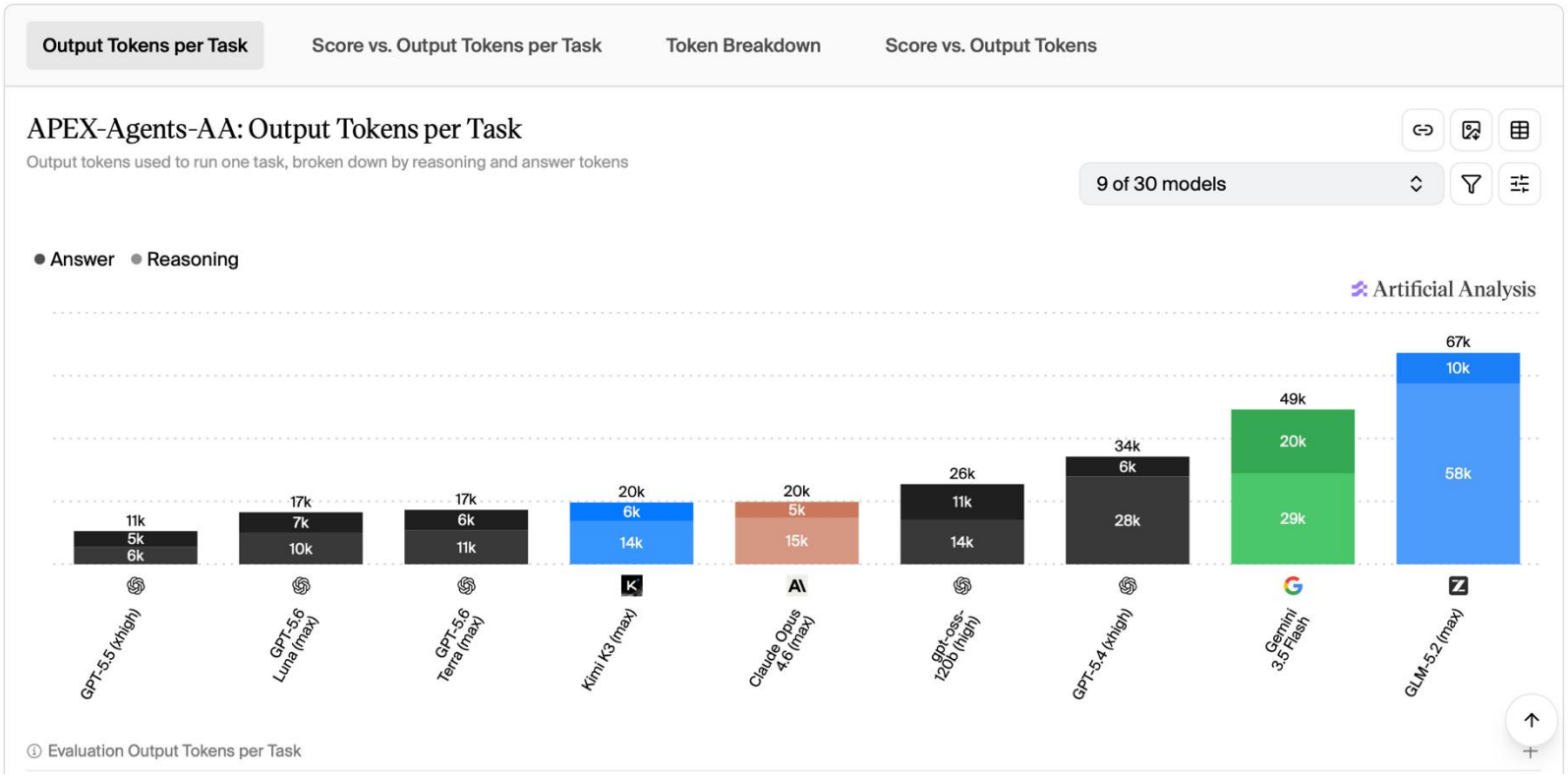
– Gibt den Vergleich zum Preis wie auch Preis vs. Intelligenz



https://artificialanalysis.ai/#price

Cost per success: Vergleich von Kosten und Performance

Token Usage



https://artificialanalysis.ai/evaluations/apex-agents-aa

Pass@k

– Diese Formel berechnet die Wahrscheinlichkeit, mindestens eine richtige Lösung zu erhalten, wenn aus n Versuchen zufällig k Lösungen ausgewählt werden.

$$\text{Pass@k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

Where:

- n is the total number of attempts
- c is the number of correct solutions
- $\binom{n}{k}$ represents the binomial coefficient (n choose k)

Frage: wir haben Evaluationen:

– 60% pass@1

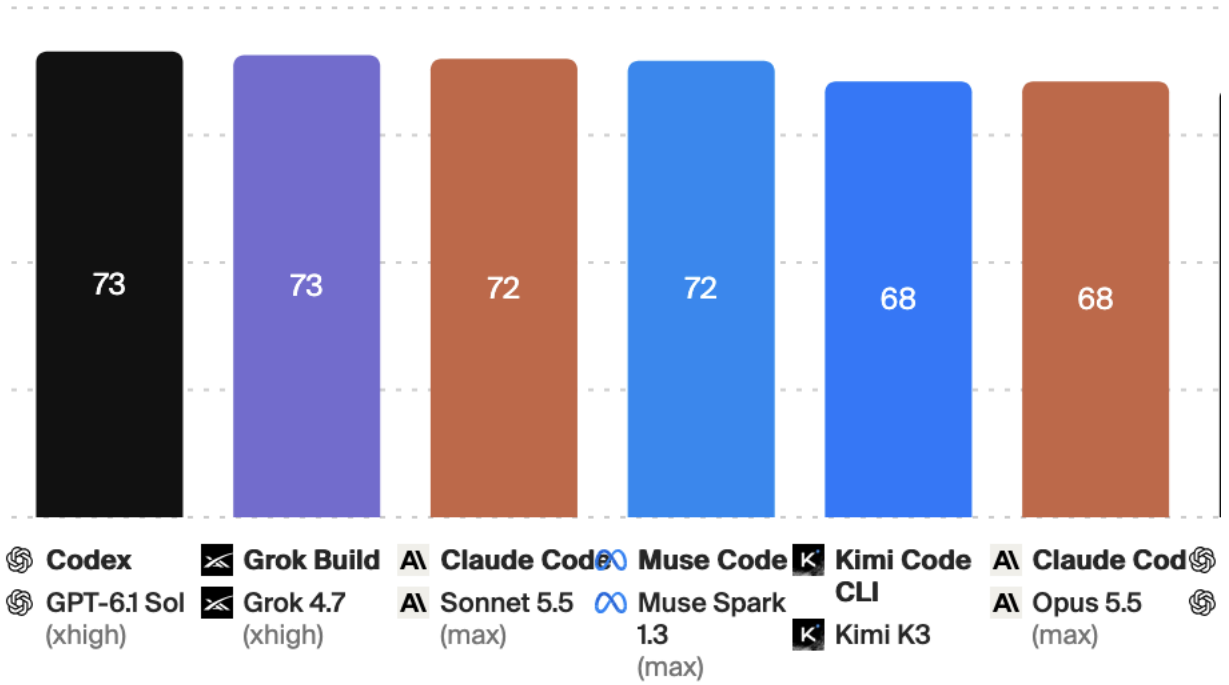
– 90% pass@10

Ist es ein 60% oder ein 90% Model?

https://ipshita.medium.com/pass-k-a-practical-metric-for-evaluating-ai-generated-code-18462308afbd

DeepSWE v1.1 Benchmark Score

Average pass@1 on DeepSWE v1.1 · Higher is better



1. Modellauswahl (3/3): Anbieterübersicht

Anbieter	API-Typ	Free-Tier ohne Kreditkarte	Typische Free-Kontingente	Besonderheiten
Google AI Studio (Gemini)	LLM + Embeddings	Ja	Limits nur im AI Studio sichtbar; Embeddings gratis	Starke multilinguale Modelle, kein Billing nötig
Cerebras Inference API	LLM	Nein (Karte nötig, seit 07/2026)	Früher 1 Mio. Tokens/Tag, jetzt Trial-Guthaben	Sehr hohe Tokenrate (2'600+ T/s), OpenAI-kompatibel
Groq	LLM	Ja	~1'000 Requests / 200k Tokens pro Tag und Modell	Niedrige Latenz, gut für Echtzeit
OpenRouter	LLM + Embeddings	Ja	19 Free-Modelle: 50 Req./Tag (1'000 ab 10 $ Guthaben)	Eine API für 200+ Modelle, wechselnder Free-Pool
Hugging Face	LLM + Embeddings	Ja, nur Guthaben	0.10 $ Guthaben/Monat (PRO: 2 $)	Keine Free-Modelle mehr; Router zu 14+ Anbietern ohne Marge
LiteLLM	Proxy / Gateway	JA (FHNW)		
Swiss AI Research Platform	LLM	Ja		

Was ist ein LLM-Gateway? Beispiel: LiteLLM

LiteLLM

Open-Source-KI-Gateway (Proxy) zwischen deinen Tools und den Modellanbietern.

Eine API für über 100 LLM-Anbieter – OpenAI-kompatibel, dazu ein Anthropic-kompatibler Endpunkt (z. B. für Claude Code).

Dein Tool kennt nur die Gateway-URL und deinen persönlichen Schlüssel.

Einheitliche API
Modelle wechseln, ohne das Tool anzupassen

Virtuelle Schlüssel
ein Schlüssel pro Person oder Team, widerrufbar

Budgets & Rate-Limits
pro Schlüssel, Nutzer oder Team

Routing & Fallbacks
Lastverteilung, erneuter Versuch bei anderem Anbieter

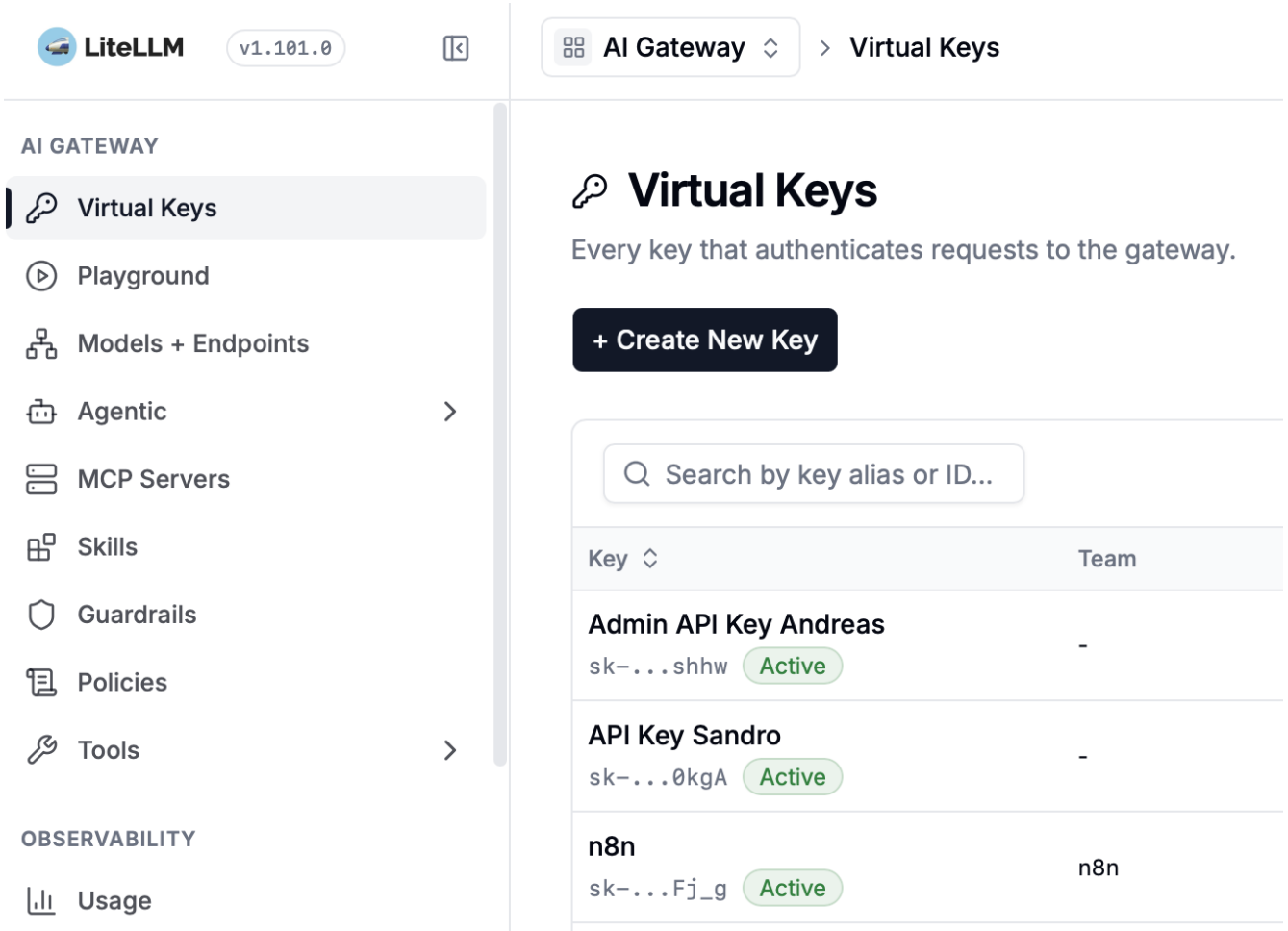
Kostenkontrolle & Logging
wer welches Modell zu welchen Kosten genutzt hat

Admin-Oberfläche
Schlüssel, Modelle und Budgets verwalten

https://docs.litellm.ai/docs/simple_proxy ; https://code.claude.com/docs/en/llm-gateway

LiteLLM

- Proxy für Sprachmodelle
- Wir (FHWN) stellen euch die Modelle zur Verfügung
- Es hat einen Kosten Maximum eingerichtet (zur Sicherheit)
- Ihr könnt alle zur Verfügung stehenden Modelle benutzen
- Ihr solltet euch über den Einladungslink mit eurem FHNW Login einloggen können.
- Bei Fragen kommt auf mich zu.



1) Proprietary vs. open model

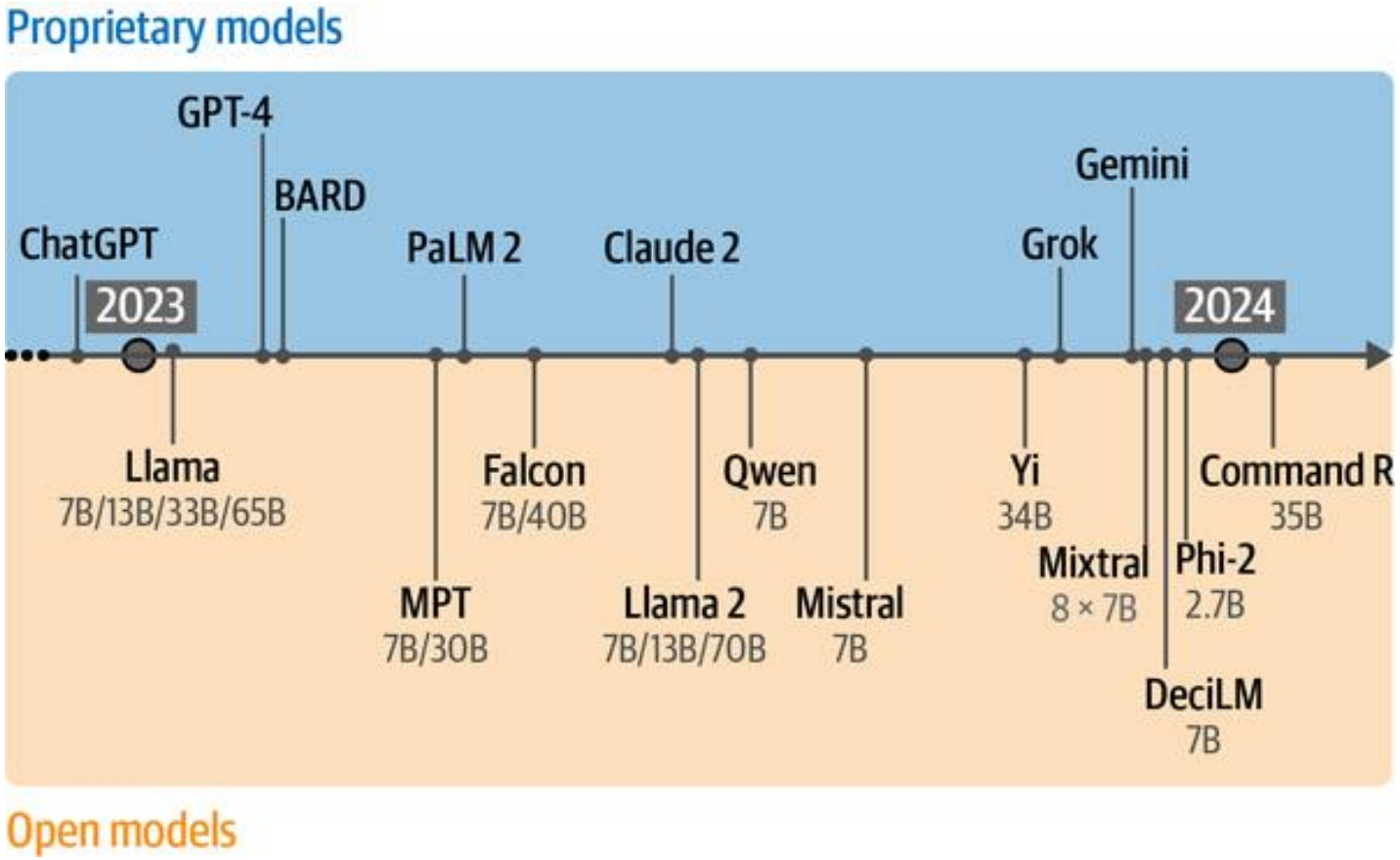


Image from Alammar, J., & Grootendorst, M. (2024). Hands-On Large Language Models: Language Understanding and Generation (First edition.). O'Reilly Media.

Grad der Offenheit

Proprietäre Modelle:

\+
- Benötigt keine GPU Ressourcen
- Hosting und Betrieb, Support/SLA sichergestellt
- Tendenziell an der „forefront“, schnelle Markteinführung

−
- Kosten
- Keine Optimierungsmöglichkeiten
- Datenweitergabe
- „Vendor Lock-in“/-Abhängigkeit

Open Source

- Komplette Kontrolle über das Model
- Sensitive Daten können ohne Bedenken verwendet werden, auch für Intellectual Property (e.g. Code)
- Optimierung, Fine-tuning etc. möglich
- Regulatorische Anforderungen existieren

- Know-How erforderlich
- Betriebsverantwortung
- Computing Power kostet auch

Dokumentation: Architektur, Paper etc.

Availability: Lizenzierung

Availability: RLHF data and weights

Availability: Training data, pre-training weights

Availability: Source code

Access: Ist das Model frei verfügbar? Kann ich es lokal betreiben?

Alammar, J., & Grootendorst, M. (2024). Hands-On Large Language Models: Language Understanding and Generation (First edition.). O'Reilly Media., https://opening-up-chatgpt.github.io

> Demo

1. Demo auf Moodle zum Download

> Aufgabe

1. Registriert euch auf der Swiss AI Research Platform
2. Loggt euch in Litellm ein
3. Generiert jeweils einen API Key
4. Ladet die Demo herunter. Benutzt die beiden API Keys und tested aus
    1. Welche Modelle stehen zur Verfügung?
    2. Was ist der Output?
    3. Funktioniert alles? Versteht ihr alles?

Zeit: 30 Min.

> Menti

Go to

www.menti.com

Enter the code

4689 1571



Or use QR code

> Aufgabe

1. Geht auf Deeplearning.ai
2. Schaut euch das Video “AI Agents in LangGraph“ an
3. Genaue Kapitel: Siehe Moodle